Vendredi 30/10 — GitHub et journal de veille

Kevin Bardoux

vendredi 30 octobre 2026

Table des matières

Objectifs 2
Veille — Les mots de passe (30 min) 2
    En groupe (15 min) 2
    Mise en commun et trace écrite (15 min) 3
Matin — GitHub : branches, Pull Requests & conflits · mise en place du journal de veille 3
    Rappel (5 min) 3
    Ajouter sa clé SSH (10 min) 3
    Publier le Git Journal (15 min) 5
        Pas à pas — lire la page d'un dépôt (5 min) 6
    Apport 1 — Branches et Pull Requests (10 min) 8
    Branches — attention et vérification 11
    Exercice — Ma première Pull Request (25 min) 11
    Apport 2 — Les conflits (10 min) 13
    Conflits — attention et vérification 15
    Mise en place du journal de veille (15 min) 15
    À retenir — ce matin 15
        Si le temps le permet — Provoquer et résoudre un conflit 16
    Débrief 17
Après-midi — Groupe B : projet & consolidation 17
    Au programme 17
    Pour aller plus loin 17
    Sources 18

> **Note**
>
> **Compétences visées** : C1 · apprendre en continu (veille, anglais) **Livrable** : une Pull Request fusionnée dans `git-journal` et le dépôt privé `journal-de-veille` avec ses premières entrées.

Objectifs

- Citer trois recommandations pour un mot de passe solide.
- Publier un dépôt sur GitHub, par SSH.
- Proposer une modification par une Pull Request, puis la fusionner.
- Reconnaître et résoudre un conflit de fusion.
- Tenir son journal de veille en ligne.

Comment se passe la matinée. Après la veille : la clé SSH et la publication (des procédures, étape par étape), puis deux notions, la branche avec la Pull Request et le conflit. Chacune suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification ; un encadré « À retenir » termine la matinée.

Veille — Les mots de passe (30 min)

En groupe (15 min)

1. **Groupe 1** : les recommandations de la CNIL.
2. Groupe 1 : longueur, caractères, phrases de passe ?
3. **Groupe 2** : les recommandations de l'ANSSI.
4. Groupe 2 : que conseille-t-elle en plus du mot de passe ?
5. Dans chaque groupe : un lecteur, un secrétaire, un porte-parole.

Sources : la page de la CNIL sur les mots de passe (CNIL, s. d.) et les recommandations de l'ANSSI sur l'authentification multifacteur et les mots de passe (ANSSI, s. d.). Si vous préférez ne pas prendre la parole devant le groupe, prenez le rôle de lecteur ou de secrétaire.

Mise en commun et trace écrite (15 min)

1. Chaque porte-parole présente ce que son groupe a trouvé (3 min).
2. Seul : rédigez votre entrée de veille sur papier.
3. Elle ouvrira votre journal en ligne, en fin de matinée.

Matin — GitHub : branches, Pull Requests & conflits · mise en place du journal de veille

Rappel (5 min)

1. Différence entre `git add` et `git commit` ?
2. Que fait `git restore` ?
3. Pourquoi renvoyer un nouveau tableau ?

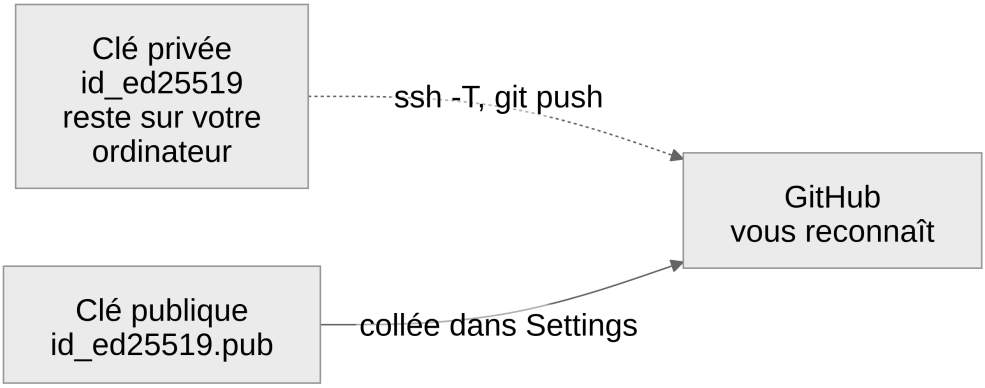
Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Ajouter sa clé SSH (10 min)

Niveau : guidé.

1. Git Bash : `cat ~/.ssh/id_ed25519.pub` ; copiez la ligne affichée.
2. GitHub : *Settings*, *SSH and GPG keys*, *New SSH key* ; collez.
3. Git Bash : `ssh -T git@github.com` ; répondez `yes` la première fois.
4. **Réussi si** : « Hi pseudo! You've successfully authenticated… ».

Ajout de la clé (GitHub, s. d.-b) et test de la connexion (GitHub, s. d.-g) : le message complet est « Hi USERNAME! You've successfully authenticated, but GitHub does not provide shell access. », avec votre pseudo à la place de `USERNAME`.



La clé privée reste sur votre ordinateur ; la clé publique est déposée sur GitHub, qui s'en sert pour vous reconnaître.

Vous devez voir, à l'étape 1 : une seule ligne, qui commence par `ssh-ed25519` et se termine par votre adresse e-mail. C'est elle que vous copiez, en entier.

```
stagiaire@PC MINGW64 ~
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAFw5W5cl1UTolfCXRc5Qy+VZoms90T6P9ur80QI3cZE
prenom.nom@exemple.fr
```

À l'étape 3, la première fois, Git Bash demande s'il peut faire confiance au serveur :

```
stagiaire@PC MINGW64 ~
$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.121.3)'
can't be established.
ED25519 key fingerprint is SHA256:
+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/
[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to
the list of known hosts.
```

1. L'adresse IP entre parenthèses peut être différente : GitHub a plusieurs serveurs.

2. Avant de taper `yes`, comparez l'empreinte (`SHA256:+DiY…`) avec celle que publie GitHub (GitHub, s. d.-d) : si elle est identique, c'est bien GitHub.
3. Puis Git Bash demande la phrase de passe de votre clé (`Enter passphrase for key…`), et GitHub vous salue par votre pseudo.
4. `Permission denied (publickey)` à la place du salut : la clé n'est pas encore ajoutée sur GitHub, ou ce n'est pas la bonne. Recommencez l'étape 2.

Publier le Git Journal (15 min)

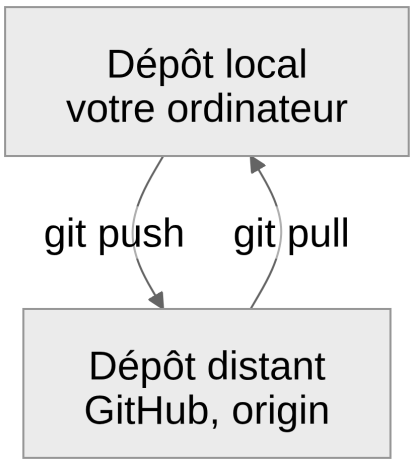
Niveau : guidé.

1. GitHub : créez un dépôt **vide** `git-journal`.
2. Git Bash : `git remote add` (commande dans le support).
3. Puis `git push -u origin main`.
4. **Réussi si** : la page du dépôt montre vos commits.

Dans Git Bash, depuis `dwwm/git-journal`, en remplaçant `pseudo` par le vôtre :

```
git remote add origin git@github.com:pseudo/git-journal.git
git push -u origin main
```

Créez le dépôt sans README, licence ni `.gitignore`, pour éviter les erreurs au premier envoi (GitHub, s. d.-c). `git remote add` associe votre dépôt local au dépôt distant ; `git push` y envoie vos commits (Scott Chacon et Ben Straub, s. d.-b).



Le même historique, à deux endroits : git push envoie vos commits sur GitHub, git pull récupère ceux de GitHub.

Vous devez voir : rien après `git remote add`. Après `git push`, Git demande la phrase de passe de votre clé, puis compte et envoie vos commits (les nombres dépendent de votre journal) :

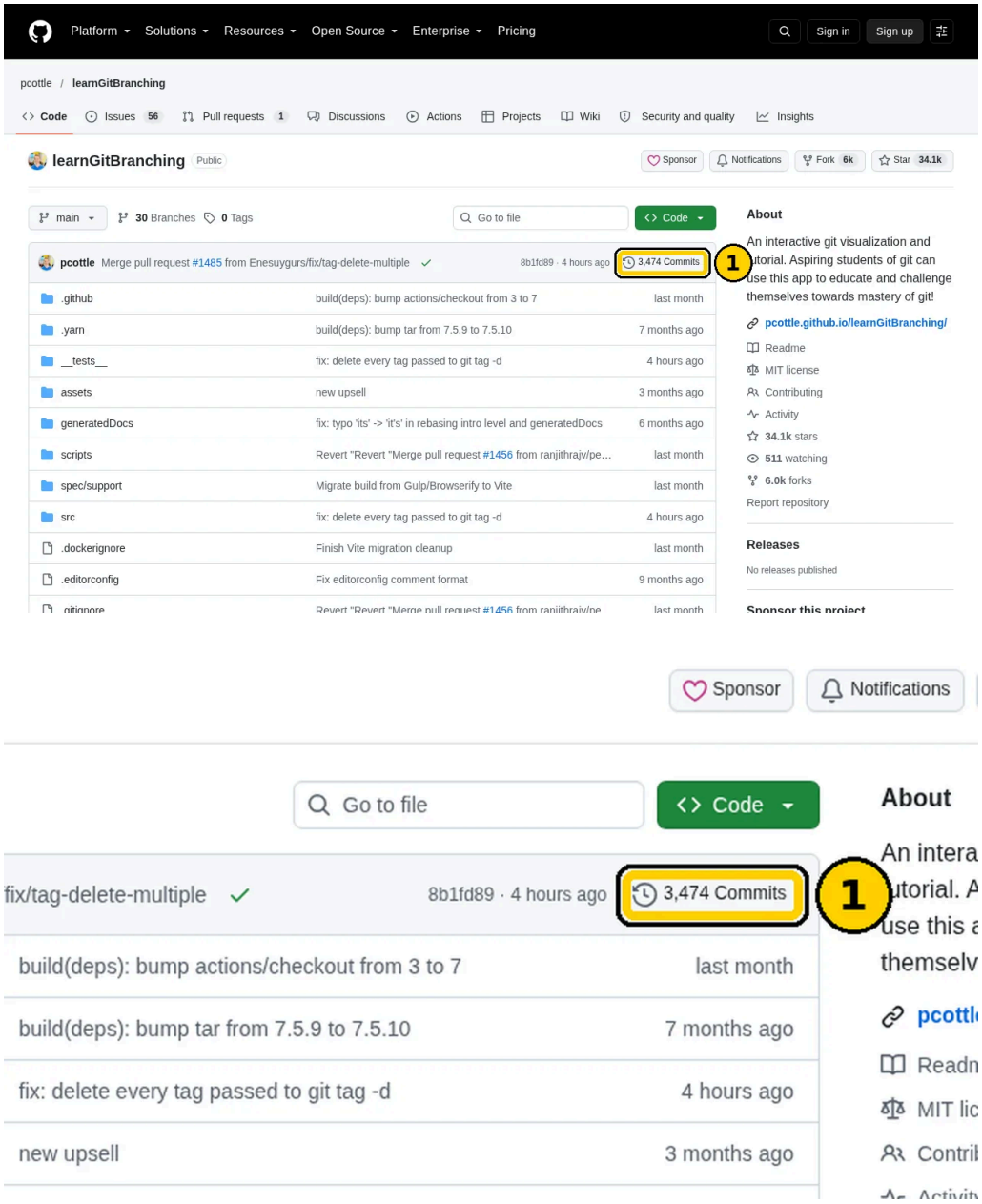
```
stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git remote add origin git@github.com:pseudo/git-
journal.git

stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git push -u origin main
Énumération des objets: 35, fait.
Décompte des objets: 100% (35/35), fait.
Compression par delta en utilisant jusqu'à 8 fils
d'exécution
Compression des objets: 100% (26/26), fait.
Écriture des objets: 100% (35/35), 3.10 Kio | 264.00
Kio/s, fait.
Total 35 (delta 9), réutilisés 0 (delta 0), réutilisés
du pack 0
To github.com:pseudo/git-journal.git
 * [new branch]      main -> main
la branche 'main' est paramétrée pour suivre 'origin/
main'.
```

La ligne `* [new branch] main -> main` dit que la branche `main` existe maintenant sur GitHub. Rechargez la page du dépôt : vos commits y sont.

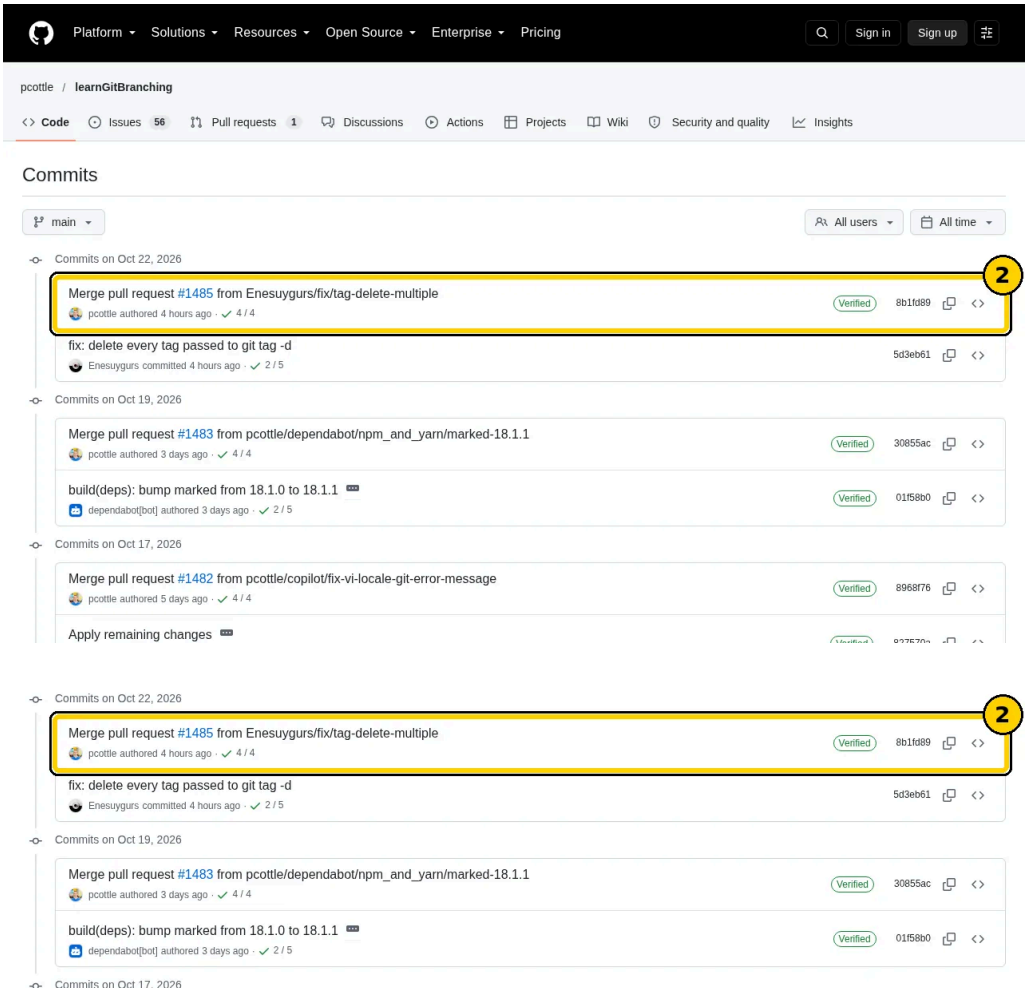
Pas à pas — lire la page d'un dépôt (5 min)
Les captures montrent un dépôt public, celui de Learn Git Branching (licence MIT) (Peter Cottle et contributeurs, s. d.) ; votre `git-journal` a la même présentation, avec moins de commits. L'endroit à cliquer ou à lire est entouré d'un cadre jaune, avec le numéro de l'étape ; un zoom suit chaque capture.

Étape 1. Sur la page du dépôt, cliquez sur le lien **Commits**, à droite du dernier commit, au-dessus de la liste des fichiers.



Vous devez voir : une page intitulée **Commits**, qui liste les commits du plus récent au plus ancien, groupés par jour.

Étape 2. Lisez le premier commit de la liste.



Vous devez voir : pour chaque commit, son message, son auteur, sa date et son identifiant court, comme dans `git log --oneline`.

Capture : dépôt public pcottle/learnGitBranching, licence MIT ; noms visibles : ceux des auteurs du projet.

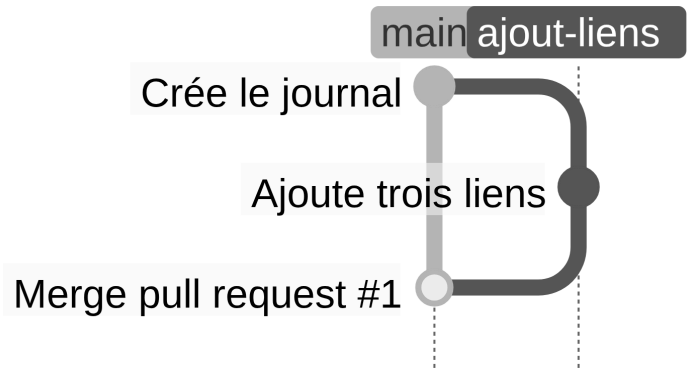
Apport 1 — Branches et Pull Requests (10 min)

- **Pourquoi** : essayer une modification sans risquer `main`.
- **Pull Request** : proposer de fusionner une branche, après relecture.

Pourquoi. `main` contient la version qui marche. Pour ajouter quelque chose, on travaille à part ; quand c'est prêt, on le propose, quelqu'un relit, puis on fusionne. C'est ainsi que vous travaillerez en binôme sur le projet #3.

Définition. Une **branche** est une ligne de travail à part : un pointeur vers un commit, créer une branche ne copie rien (Scott Chacon et Ben Straub, s. d.-a). Une **Pull Request**, sur GitHub,

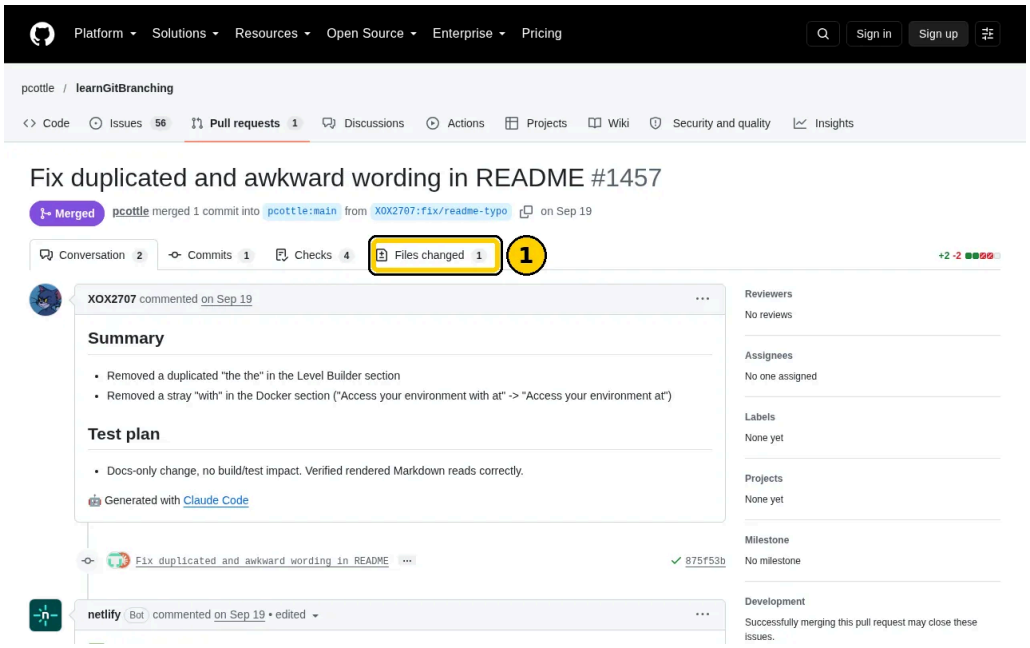
présente les modifications d'une branche pour les faire relire, puis les fusionner dans `main` (GitHub, s. d.-e).

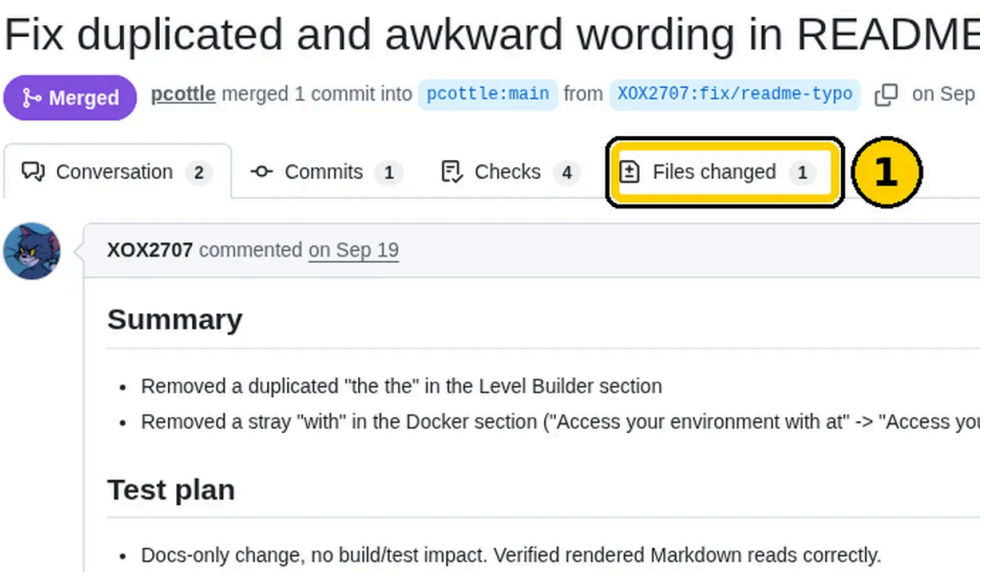


Une branche, puis sa fusion (schéma simplifié de l'exercice) : le commit est fait sur ajout-liens, main ne change pas ; la Pull Request acceptée crée un commit de fusion sur main.

Une Pull Request sur GitHub. Les deux captures montrent une Pull Request déjà fusionnée d'un dépôt public : une correction de deux fautes dans un README (XOX2707, s. d.).

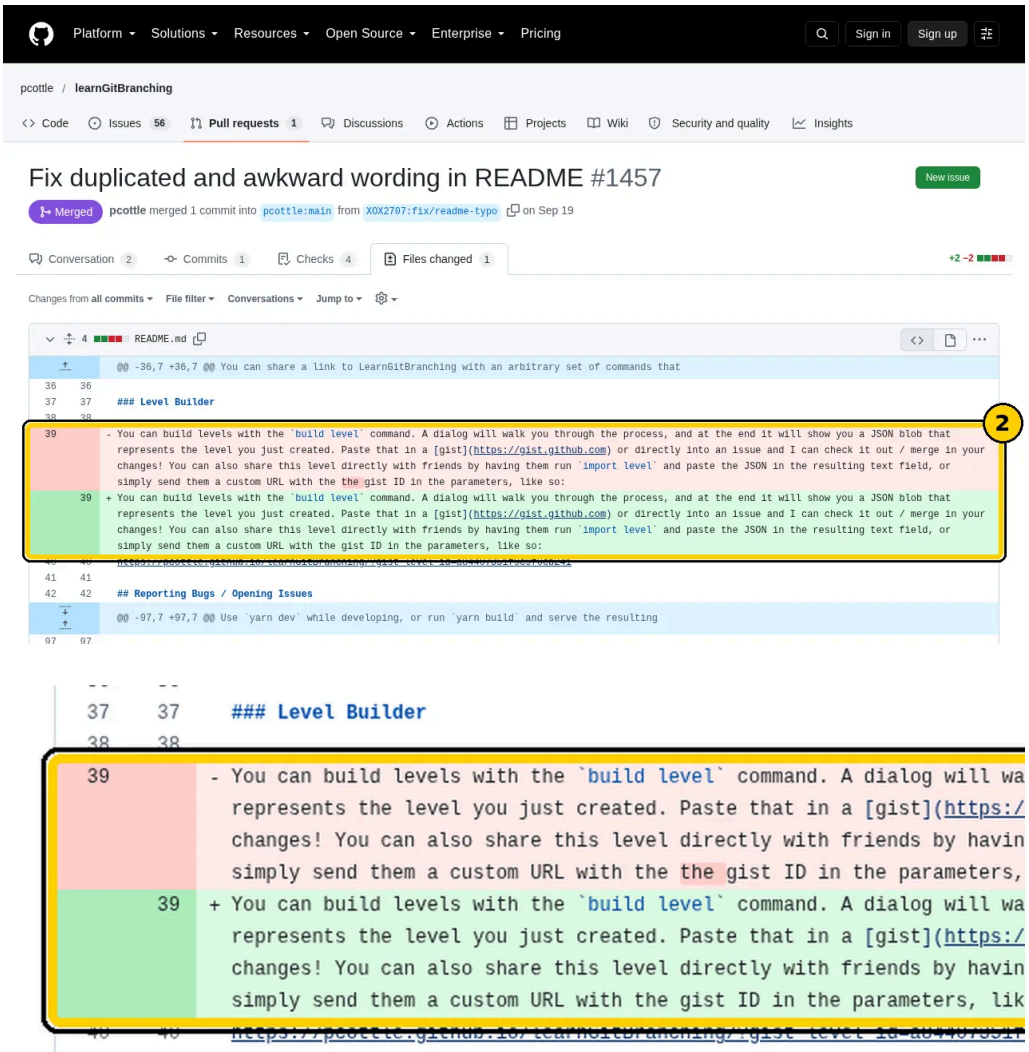
Étape 1. En haut de la Pull Request : son titre, le badge violet **Merged** (fusionnée), puis les onglets. Cliquez sur l'onglet **Files changed**.





Vous devez voir : le fichier modifié, `README.md`, ligne par ligne.

Étape 2. Lisez les deux lignes de couleur.



Vous devez voir : en rouge (signe `-`), la ligne retirée ; en vert (signe `+`), la ligne ajoutée. C'est ce que votre binôme relira dans vos Pull Requests.

Capture : dépôt public pcottle/learnGitBranching, licence MIT ; noms visibles : ceux des auteurs de la Pull Request.

Exemple.

```
git switch -c ajout-liens          # créer la branche et
s'y placer
git add journal.md
git commit -m "Ajoute trois liens de documentation"
git push -u origin ajout-liens     # envoyer la branche
sur GitHub
```

Pas à pas. C'est l'exercice ci-dessous : le formateur fait chaque étape au vidéoprojecteur, vous la refaites aussitôt.

Branches — attention et vérification

> ⚠ Attention
>
> Avant de modifier, vérifiez la branche où vous êtes : l'invite de Git Bash l'affiche entre parenthèses, et `git status` en première ligne (`Sur la branche …`, ou `On branch …` si Git s'affiche en anglais).

- **Vérifiez** : quelle commande crée une branche et s'y place ?

Exercice — Ma première Pull Request (25 min)

Niveau : guidé.

1. Ensemble : créez la branche `ajout-liens` dans `git-journal`.
2. Ajoutez à `journal.md` trois liens de documentation ; commitez.
3. Poussez la branche ; sur GitHub, *Compare & pull request*.
4. Fusionnez (*Merge*) ; puis `git switch main` et `git pull`.
5. **Réussi si** : `git log --oneline` sur `main` montre vos liens.

Vous devez voir, aux étapes 1 à 3 (créer la branche, enregistrer, envoyer) :

```
stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git switch -c ajout-liens
Basculement sur la nouvelle branche 'ajout-liens'

stagiaire@PC MINGW64 ~/dwwm/git-journal (ajout-liens)
$ git add journal.md

stagiaire@PC MINGW64 ~/dwwm/git-journal (ajout-liens)
$ git commit -m "Ajoute trois liens de documentation"
[ajout-liens 7b1ddb3] Ajoute trois liens de
documentation
 1 file changed, 6 insertions(+)

stagiaire@PC MINGW64 ~/dwwm/git-journal (ajout-liens)
$ git push -u origin ajout-liens
…
To github.com:pseudo/git-journal.git
 * [new branch]      ajout-liens -> ajout-liens
la branche 'ajout-liens' est paramétrée pour suivre
'origin/ajout-liens'.
```

Le … remplace les lignes de comptage, comme pour `main`. Après `git switch -c`, l'invite affiche `(ajout-liens)` : vous travaillez sur la branche.

À l'étape 4, après la fusion sur GitHub :

```
stagiaire@PC MINGW64 ~/dwwm/git-journal (ajout-liens)
$ git switch main
Votre branche est à jour avec 'origin/main'.
Basculement sur la branche 'main'

stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git pull
Mise à jour 7911617..05e9b1c
Fast-forward
 journal.md | 6 ++++++
 1 file changed, 6 insertions(+)
```

```
stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git log --oneline -3
05e9b1c Merge pull request #1 from pseudo/ajout-liens
7911617 Ajoute un lien vers Pro Git
7b1ddb3 Ajoute trois liens de documentation
```

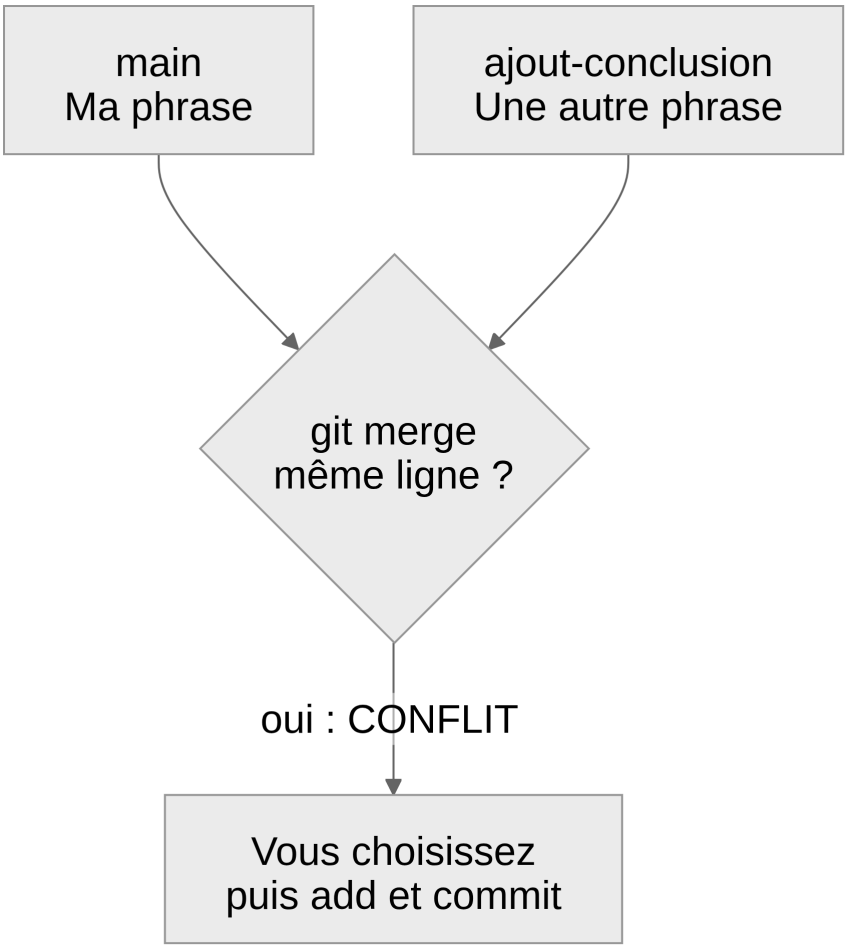
`git pull` récupère la fusion faite sur GitHub : le commit `Merge pull request #1…` est maintenant aussi sur votre ordinateur. `Votre branche est à jour avec 'origin/main'` veut dire « à jour avec ce que Git savait de GitHub avant le `pull` », pas forcément avec GitHub : c'est pour cela que l'on fait `git pull` ensuite.

Apport 2 — Les conflits (10 min)

- **Pourquoi** : deux modifications de la même ligne, Git ne choisit pas.
- **Conflit** : des marqueurs dans le fichier ; **vous** décidez.

Pourquoi. Quand deux branches modifient la même ligne, Git ne sait pas quelle version garder : il s'arrête et vous demande de choisir.

Définition. Un conflit de fusion est signalé par des marqueurs dans le fichier : les modifications de la branche courante après `<<<<<<< HEAD`, celles de l'autre branche après `=======`, jusqu'à `>>>>>>>` (GitHub, s. d.-f).



Deux branches modifient la même ligne : git merge s'arrête sur un conflit, et c'est vous qui choisissez la version à garder.

Exemple, avant et après. Avant : le fichier tel que Git le laisse, avec les deux versions entre les marqueurs.

```
<<<<<<< HEAD
Ma phrase, sur la branche main.
=======
Une autre phrase, sur la branche ajout-conclusion.
>>>>>>> ajout-conclusion
```

Après : vous avez gardé la phrase de `main` et effacé les trois lignes de marqueurs. Il ne reste que le texte voulu :

```
Ma phrase, sur la branche main.
```

Pas à pas. Le formateur fait la démonstration en direct ; vous la refaites dans « Si le temps le permet », ou cet après-midi.

1. Ouvrez le fichier en conflit dans VS Code.

2. Lisez les deux versions, entre les marqueurs.
3. Gardez le bon texte ; effacez les trois lignes de marqueurs.
4. Enregistrez, puis `git add journal.md` et `git commit -m "Résout le conflit de la conclusion"`.

La fiche Lire un message d'erreur montre le message de Git qui annonce un conflit, et celui d'un envoi refusé.

Conflits — attention et vérification

> ⚠ Attention
>
> Un marqueur oublié (`<<<<<<<`, `=======`, `>>>>>>>`) reste dans le fichier et casse le code. Cherchez-les avant le commit.

- **Vérifiez** : combien de lignes de marqueurs effacer ?

Mise en place du journal de veille (15 min)

Niveau : guidé.

1. GitHub : dépôt **privé** `journal-de-veille`, avec `README.md`.
2. Reprenez le modèle d'entrée fourni dans l'archive.
3. Première entrée : les mots de passe (ce matin).
4. Ajoutez vos feuilles de veille des semaines 1 et 2.
5. *Settings*, *Collaborators* : invitez le formateur.

Un dépôt privé n'est accessible qu'à vous et aux personnes que vous invitez (GitHub, s. d.-a) : invitez le formateur pour qu'il puisse suivre votre veille. Le modèle d'entrée est dans l'archive de la semaine. À partir de maintenant, chaque veille du vendredi ajoute une entrée à ce journal.

À retenir — ce matin

> À retenir
>
> - **Branche** : `git switch -c nom` ; `main` reste intact.
> - **Pull Request** : proposer, relire, fusionner ; puis `git pull`.
> - **Conflit** : vous choisissez ; aucun marqueur ne doit rester.

Le détail, notion par notion :

- **Branches et Pull Requests** : une branche est une ligne de travail à part ; on la pousse, on ouvre une Pull Request, on la relit, on la fusionne, puis on récupère `main` avec `git pull`.
- **Conflits** : deux modifications de la même ligne ; les marqueurs montrent les deux versions ; on garde le bon texte, on efface les marqueurs, puis `git add` et `git commit`.

Si le temps le permet — Provoquer et résoudre un conflit

1. Sur `main`, modifiez la dernière ligne de `journal.md` ; commitez.
2. Créez une branche depuis le commit précédent, modifiez la même ligne.
3. Fusionnez : Git signale un conflit. Résolvez-le, puis commitez.

Pour créer la branche depuis le commit précédent : `git switch -c ajout-conclusion HEAD~1`. Pour fusionner depuis `main` : `git merge ajout-conclusion`. Suivez ensuite les étapes de résolution de la documentation de GitHub (GitHub, s. d.-f).

Vous devez voir, à l'étape 3 :

```
stagiaire@PC MINGW64 ~/dwwm/git-journal (main)
$ git merge ajout-conclusion
Fusion automatique de journal.md
CONFLIT (contenu) : Conflit de fusion dans journal.md
La fusion automatique a échoué ; réglez les conflits et
validez le résultat.
```

Puis, dans `journal.md`, les deux versions entre les marqueurs :

```
<<<<<<< HEAD
Ma conclusion, sur la branche main.
=======
Une autre conclusion, sur la branche ajout-conclusion.
>>>>>>> ajout-conclusion
```

Après la résolution, terminez par `git add journal.md`, puis `git commit -m "Résout le conflit de la conclusion"`. Sans `-m`, Git ouvrirait un éditeur dans le terminal (voir jeudi). L'invite affiche `(main|MERGING)` pendant que la fusion est en cours.

Débrief

- Pourquoi travailler sur une branche plutôt que sur `main` ?
- À quoi sert une Pull Request, même quand on travaille seul ?
- Qui décide de la version à garder lors d'un conflit ?

Critères de réussite

- ☐ `git-journal` est sur GitHub, avec une Pull Request fusionnée
- ☐ `ssh -T git@github.com` vous salue par votre pseudo
- ☐ `journal-de-veille` est privé, avec au moins trois entrées
- ☐ Le formateur est invité sur le journal de veille

Après-midi — Groupe B : projet & consolidation

> **Note**
>
> **Groupe B** en formation de 12 h 45 à 17 h ; le groupe A est au sport (13 h 15 – 16 h). Pas de notion nouvelle : on termine, on consolide, on approfondit.

Au programme

1. **Terminer** : la check-list C1, la PR, le journal de veille.
2. **Consolider** : provoquez et résolvez un conflit.
3. **Approfondir** : Learn Git Branching, séquence suivante.
4. **Relire** (16 h 15) : la PR d'un camarade, un commentaire.

Aucune notion nouvelle : branches, Pull Requests et conflits ont été vus ce matin. Le conflit provoqué suit les étapes de « Si le temps le permet », plus haut. Pour la relecture, ouvrez la Pull Request d'un camarade sur GitHub et laissez un commentaire (une ligne claire, une question) : vous le ferez souvent à partir du projet #3. Le groupe A n'en a pas besoin pour la suite : les comptes (lundi), la clé SSH et la première PR (ce matin) sont faits par tout le monde.

Pour aller plus loin

- Markdown Guide : la syntaxe de vos `README.md`, avec des exemples.

La page *Basic Syntax* de Markdown Guide présente chaque élément (titres, listes, liens, images, code) avec sa syntaxe, le HTML produit et le rendu (Matt Cone, s. d.). Gratuit, sans compte, en anglais. Mettez en forme le `README.md` de votre journal de veille : un titre, une liste, un lien.

Sources

ANSSI. s. d. « Recommandations relatives à l'authentification multifacteur et aux mots de passe ». Consulté le 26 septembre 2026. https://cyber.gouv.fr/publications/recommandations-relatives-lauthentification-multifacteur-et-aux-mots-de-passe.

CNIL. s. d. « Mots de passe : recommandations pour maîtriser sa sécurité ». Consulté le 26 septembre 2026. https://www.cnil.fr/fr/mots-de-passe-des-recommandations-de-securite-minimales-pour-les-entreprises-et-les-particuliers.

GitHub. s. d.-a. « About repositories ». Consulté le 26 septembre 2026. https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories.

GitHub. s. d.-b. « Adding a new SSH key to your GitHub account ». Consulté le 26 septembre 2026. https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account.

GitHub. s. d.-c. « Adding locally hosted code to GitHub ». Consulté le 26 septembre 2026. https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github.

GitHub. s. d.-d. « GitHub's SSH key fingerprints ». Consulté le 17 octobre 2026. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/githubs-ssh-key-fingerprints.

GitHub. s. d.-e. « Pull requests - GitHub Docs ». Consulté le 26 septembre 2026. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests.

GitHub. s. d.-f. « Resolving a merge conflict using the command line ». Consulté le 26 septembre 2026. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-using-the-command-line.

GitHub. s. d.-g. « Testing your SSH connection ». Consulté le 26 septembre 2026. https://docs.github.com/en/authentication/connecting-to-github-with-ssh/testing-your-ssh-connection.

Matt Cone. s. d. « Basic Syntax — Markdown Guide ». Consulté le 3 octobre 2026. https://www.markdownguide.org/basic-syntax/.

Peter Cottle et contributeurs. s. d. « pcottle/learnGitBranching : dépôt public sur GitHub (licence MIT) ». Consulté le 22 octobre 2026. https://github.com/pcottle/learnGitBranching.

Scott Chacon et Ben Straub. s. d.-a. « Pro Git : Les branches en bref ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-branches-avec-Git-Les-branches-en-bref.

Scott Chacon et Ben Straub. s. d.-b. « Pro Git : Travailler avec des dépôts distants ». Consulté le 26 septembre 2026. https://git-scm.com/book/fr/v2/Les-bases-de-Git-Travailler-avec-des-d%C3%A9p%C3%B4ts-distants.

XOX2707. s. d. « Fix duplicated and awkward wording in README (Pull Request #1457, pcottle/learnGitBranching) ». Consulté le 22 octobre 2026. https://github.com/pcottle/learnGitBranching/pull/1457.